

Clase 3 — Caché

- Clase 3 — Caché
 - 1. Introducción
 - 2. Qué problema resuelve una caché
 - 3. Modelo mental: copias, fuente de verdad y frescura
 - 4. Dónde vive la caché
 - 5. Capacidad, expiración y expulsión
 - 6. Patrones de acceso y actualización
 - 7. Invalidación y datos obsoletos
 - 8. Fallos, picos de tráfico y anti-patrones
 - 9. Observabilidad y decisión arquitectónica
 - 10. EventOS: aplicación práctica
 - 11. Trade-offs y decisiones arquitectónicas
 - 12. Puntos clave

Clase 3 — Caché

ARQ 2026 · UCC

1. Introducción

Una caché es un almacenamiento que conserva temporalmente copias de datos costosos de obtener. Su objetivo es responder algunas solicitudes sin consultar la fuente original —por ejemplo, una base de datos, otro servicio o un cálculo intensivo—. Al evitar ese acceso, puede disminuir la latencia que percibe el usuario y reducir la carga sobre una dependencia.

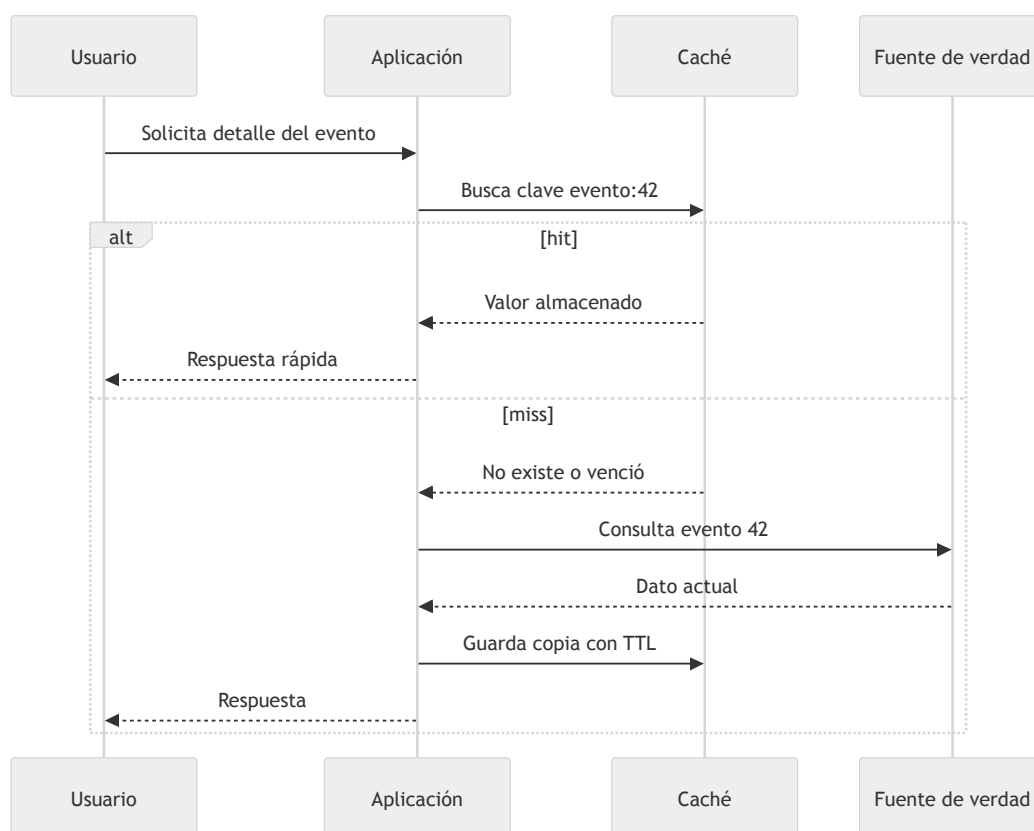
La caché no es una fuente de verdad adicional. Es una copia que puede estar ausente, vencida o desactualizada. Por eso, usarla es una decisión arquitectónica: además de mejorar lecturas, incorpora preguntas que el sistema debe responder con precisión. ¿Cuál es la fuente de verdad? ¿Cuánto tiempo puede un dato no reflejar un cambio? ¿Cómo se invalida la copia? ¿Qué ocurre si la caché deja de responder?

El propósito de este apunte es analizar esas preguntas para elegir cuándo cachear, qué tipo de caché usar y cómo operar los patrones más frecuentes sin convertir una optimización de rendimiento en una fuente de errores difíciles de detectar.

2. Qué problema resuelve una caché

Una solicitud puede ser lenta por varios motivos: trabajo de CPU, operaciones de entrada/salida, viajes de red o contención en una dependencia compartida. Una caché ayuda solo en una situación particular: cuando distintas solicitudes necesitan un resultado equivalente y volver a obtenerlo desde su fuente original tiene un costo relevante.

Por ejemplo, una página pública que muestra el detalle de un evento puede recibir miles de consultas en pocos minutos. Si el nombre, la fecha, el lugar y la imagen del evento cambian pocas veces al día, hacer una consulta a la base de datos por cada visita genera trabajo repetido. Con una caché, la mayoría de esas consultas puede responderse desde una copia temporal.



2.1 Hits, misses y tasa de aciertos

Un **hit** ocurre cuando la caché contiene una entrada válida para la solicitud. Un **miss** ocurre cuando la entrada no existe, venció o no puede utilizarse. En un miss, la aplicación todavía debe resolver la solicitud usando otra fuente.

La **tasa de aciertos** (*hit rate*) expresa qué proporción de lecturas fue respondida por la caché:

$$\text{hit rate} = \text{hits} / (\text{hits} + \text{misses})$$

Una tasa de aciertos alta suele ser buena señal, pero no es una meta suficiente por sí sola. Una caché puede tener 95 % de hits sobre consultas muy baratas y no modificar significativamente la experiencia. A la inversa,

una tasa moderada puede aportar mucho si evita consultas excepcionalmente caras durante un pico de tráfico. La métrica debe interpretarse junto con latencia, carga de la fuente de verdad y comportamiento ante fallos.

El denominador considera únicamente las lecturas que efectivamente llegaron a la capa de caché. Errores de red, *timeouts* y reintentos deben medirse por separado: mezclarlos con hits y misses ocultaría si el problema es falta de contenido o falta de disponibilidad de la caché.

2.2 El costo de un miss

Una caché no elimina la dependencia original: desplaza una parte de las lecturas hacia una copia. En cada miss, la aplicación paga el costo completo de ir a la fuente de verdad y, normalmente, también el de poblar la caché. Si las claves casi nunca se reutilizan, o si el dato cambia antes de reutilizarse, cachear añade complejidad con poco beneficio.

Por eso, los candidatos más frecuentes son datos con estas propiedades:

- se leen muchas veces;
- son iguales para muchos consumidores o se repiten para el mismo consumidor;
- son costosos de reconstruir;
- pueden tolerar una ventana definida de desactualización;
- tienen una clave estable que identifica correctamente el resultado.

No son buenos candidatos los datos cuya corrección debe ser inmediata para tomar una decisión irreversible: disponibilidad final de entradas al confirmar una compra, saldo de una cuenta, permisos revocados o el estado de un pago. Una interfaz puede mostrar una aproximación almacenada, pero la operación crítica debe validar contra la fuente que mantiene la regla de negocio.

3. Modelo mental: copias, fuente de verdad y frescura

Antes de elegir una tecnología conviene modelar el dato. La fuente de verdad es el componente autorizado para decidir su valor actual. La caché almacena una representación derivada de ese valor, identificada por una clave y válida bajo ciertas condiciones.

3.1 La frescura es un requisito de negocio

No todos los datos requieren la misma actualización. Conviene distinguir tres categorías según su aptitud para cachear:

- **Datos inmutables:** una vez creados no cambian (por ejemplo, el comprobante de una compra ya confirmada). Son el mejor candidato: se pueden cachear con TTL largo o sin vencimiento, sin riesgo de mostrar algo incorrecto.
- **Datos de cambio infrecuente:** cambian, pero con baja frecuencia frente a la cantidad de lecturas (el detalle público de un evento). Son buenos candidatos con un TTL moderado y alguna forma de

invalidación ante el cambio.

- **Datos altamente dinámicos:** cambian con una frecuencia comparable o mayor a la de lectura (la cantidad de entradas disponibles durante una venta). Cachearlos aporta poco y puede inducir a mostrar información ya vieja; conviene evitarlo o limitarlo a una aproximación no crítica.

Un catálogo de eventos puede admitir que un cambio de descripción tarde unos minutos en aparecer. La ocupación exacta de una sala durante una venta con alta demanda puede requerir validación inmediata. La pregunta correcta no es “¿cuál es el TTL estándar?”, sino “¿cuánto desfase puede observar un usuario y qué daño causa?”.

Un **TTL** (*time to live*) es el tiempo máximo durante el que una entrada se considera utilizable desde que se escribe. Al vencer, la entrada se descarta o debe renovarse. Un TTL limita el tiempo de obsolescencia, pero no garantiza que el dato sea actualizado: una entrada puede quedar vieja inmediatamente después de guardarse si la fuente de verdad cambia.

Hay dos mecanismos complementarios:

Mecanismo	Idea	Ventaja	Límite
Expiración por TTL	La entrada vence después de un tiempo	Simple y acota la antigüedad	No reacciona enseguida a un cambio
Invalidación explícita	Un cambio elimina o actualiza la clave relacionada	Puede reflejar cambios rápidamente	Requiere conocer todas las claves afectadas

En la práctica es habitual combinar ambos: invalidación ante una escritura conocida y TTL como red de seguridad frente a invalidaciones omitidas.

Conviene distinguir tres causas de que una clave deje de estar disponible:

Concepto	Qué lo dispara	Qué significa
Expiración	Se cumplió el TTL	La entrada alcanzó su vida máxima y ya no debe utilizarse.
Invalidación	Un cambio conocido en la fuente de verdad	La aplicación elimina o actualiza antes de tiempo una copia que podría haber quedado obsoleta.
Expulsión (<i>eviction</i>)	Presión de capacidad o memoria	El motor de caché descarta una entrada según una política como LRU, LFU o FIFO, aunque su TTL no haya vencido.

Los tres casos suelen producir un miss en la aplicación, pero tienen causas y correcciones distintas. Un TTL mal elegido se revisa desde el requisito de frescura; una invalidación ausente exige revisar el flujo de escritura; una expulsión frecuente exige revisar capacidad, tamaños y política de memoria.

3.2 Las claves forman parte del contrato

La clave de caché debe incluir todos los datos que hacen diferente a una respuesta. Para un detalle público puede bastar `eventos:42:v1`; para una respuesta que depende de idioma, moneda y rol del usuario, esos parámetros deben estar representados. Usar `eventos:42` para cachear una respuesta personalizada puede devolver información de un usuario a otro.

Una convención de claves legible ayuda a operarla y a invalidarla:

```
eventos:{id}:publico:v1
eventos:{id}:publico:es-AR:v1
usuarios:{id}:perfil:v2
```

El sufijo de versión permite abandonar representaciones antiguas sin intentar transformarlas dentro de la caché. No sustituye una política de expiración: las versiones que ya no se consultan deben vencer o eliminarse para no consumir memoria indefinidamente.

3.3 Caché no equivale a una transacción distribuida

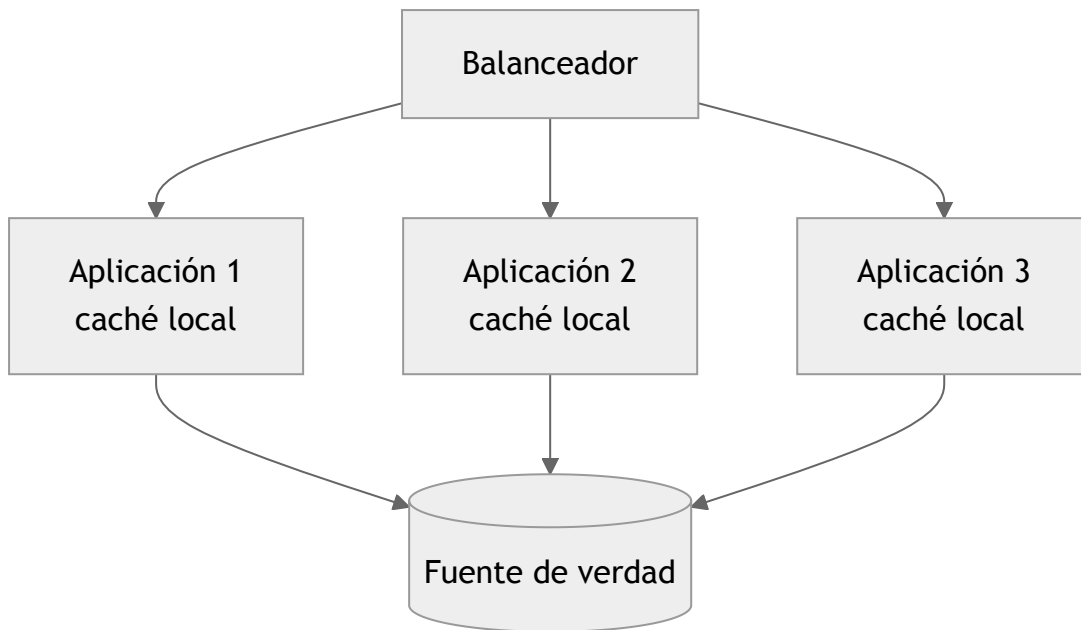
La existencia de una copia adicional no transforma una escritura en atómica. Si una operación actualiza la base y después actualiza la caché, cualquiera de los dos pasos puede fallar. El diseño debe definir qué estado queda visible y cómo se repara. Para muchos casos de lectura, borrar la clave después de confirmar la escritura en la fuente de verdad es más simple y seguro que intentar mantener ambas copias sincronizadas en cada paso.

4. Dónde vive la caché

4.1 Caché local

Una caché local reside en la memoria del mismo proceso que procesa la solicitud. No necesita un salto de red y suele ser la alternativa de menor latencia. También es simple de incorporar para valores pequeños y de vida corta.

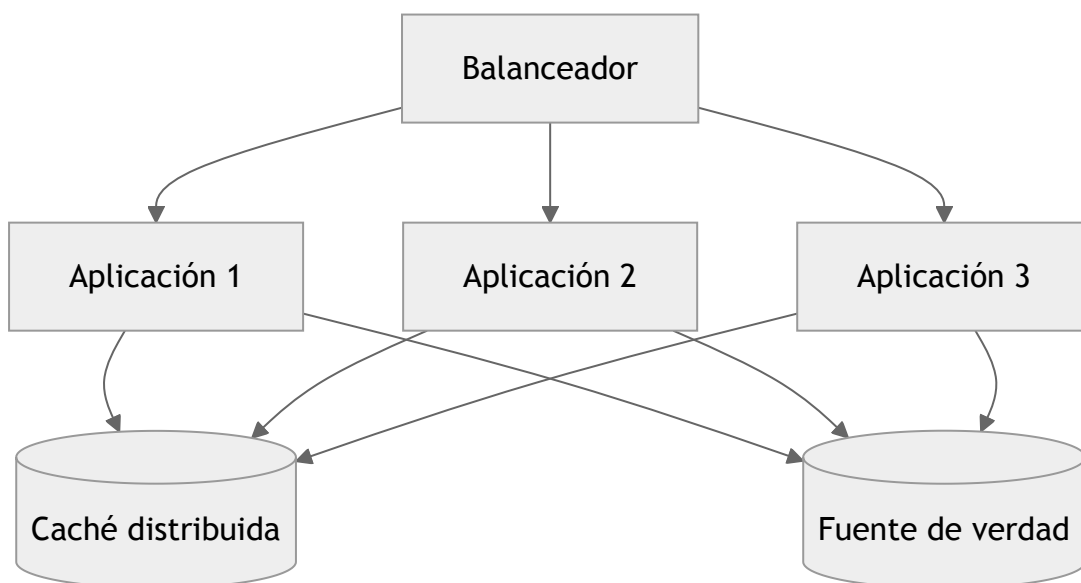
Su límite principal aparece al escalar horizontalmente. Si una aplicación tiene cuatro réplicas, cada una conserva su propia caché. Una réplica puede tener el dato actualizado y otra no; una puede recibir todo el tráfico que necesita una clave y otra puede sufrir un miss. Además, un reinicio vacía su contenido.



Una caché local resulta apropiada cuando el desfase entre instancias es admisible, el valor se puede reconstruir fácilmente o el beneficio de evitar una dependencia de red supera los costos de mantener copias separadas. Ejemplos típicos: configuraciones poco cambiantes, metadatos de corta vida o resultados computados dentro del proceso.

4.2 Caché distribuida

Una caché distribuida es un servicio al que acceden varias réplicas de una aplicación —y, si el diseño lo permite, varios componentes—. Así, un valor poblado por una instancia puede reutilizarse desde otra. Esto mejora la tasa de aciertos global y permite centralizar expiraciones.



El beneficio tiene costo: serialización, red, límites de capacidad y una dependencia operacional más. Una llamada a la caché puede tardar más que una consulta a memoria local; solo es útil frente a una alternativa aún más costosa. También puede fallar o degradarse, por lo que las solicitudes deben tener timeouts y un comportamiento de degradación definido.

4.3 Caché en dos niveles

Un diseño puede combinar ambas: una capa local para los valores más frecuentes y una distribuida para compartir valores entre réplicas. La aplicación busca primero en memoria local, luego en la distribuida y finalmente en la fuente de verdad.

Este esquema reduce aún más la latencia de los hits, pero amplifica el problema de invalidación: una actualización debe contemplar copias locales existentes. Debe elegirse solo si la ganancia de latencia justifica esa complejidad; no es una evolución obligatoria de toda caché distribuida.

En particular, invalidar una clave en L2 no elimina por sí solo las copias que ya están en la L1 de cada réplica. Un TTL breve en L1 puede acotar esa divergencia, pero incrementa lecturas a L2; otras opciones son propagar invalidaciones o prescindir de L1 para ese dato. La alternativa apropiada depende de la frescura requerida y del beneficio de evitar la llamada a L2.

4.4 Redis como caché distribuida

Redis es un almacén de datos en memoria que permite guardar valores por clave, asignar expiraciones y trabajar con varias estructuras de datos. Por esas capacidades se utiliza con frecuencia como caché distribuida.

Redis no decide qué datos son cacheables, cuál es el TTL correcto ni cómo invalidar una representación. Es infraestructura que ejecuta una política definida por la aplicación. También requiere decisiones operativas: límites de memoria, política de expulsión, persistencia si se usa para otros propósitos, autenticación, observabilidad y alta disponibilidad según el riesgo aceptado.

Para uso de caché, las operaciones conceptuales principales son:

```
GET clave           # recuperar una entrada
SET clave valor EX 60 # guardar una entrada que vence en 60 segundos
DEL clave           # invalidar una entrada
```

El valor normalmente se serializa. Guardar documentos demasiado grandes, o serializar y deserializar objetos costosos en cada acceso, puede anular buena parte del beneficio. Cachear es una optimización de un camino completo, no solo de una consulta a base de datos.

5. Capacidad, expiración y expulsión

Un TTL controla cuándo una entrada deja de ser válida desde el punto de vista del dato. No garantiza que la caché tenga memoria suficiente para retenerla. Cuando el espacio es limitado, una política de **expulsión**

(*eviction*) decide qué entradas descartar para hacer lugar a otras.

Política	Descarta primero	Cuándo suele rendir mejor	Riesgo
LRU (<i>Least Recently Used</i>)	Lo menos usado recientemente	Accesos con localidad temporal	Un dato importante pero inactivo por poco tiempo puede salir
LFU (<i>Least Frequently Used</i>)	Lo menos usado en frecuencia	Claves populares y estables	Puede conservar popularidad histórica que ya no representa el tráfico actual
FIFO (<i>First In, First Out</i>)	Lo que ingresó primero	Política simple y predecible	No considera uso ni frecuencia

TTL y expulsión resuelven problemas distintos. Una entrada puede ser válida pero expulsarse por presión de memoria; una entrada puede seguir ocupando memoria hasta que expire aunque nadie vuelva a consultarla. Es importante medir evicciones: un aumento puede señalar que el tamaño asignado es insuficiente, que se guardan valores demasiado grandes o que la carga de claves no tiene reutilización suficiente.

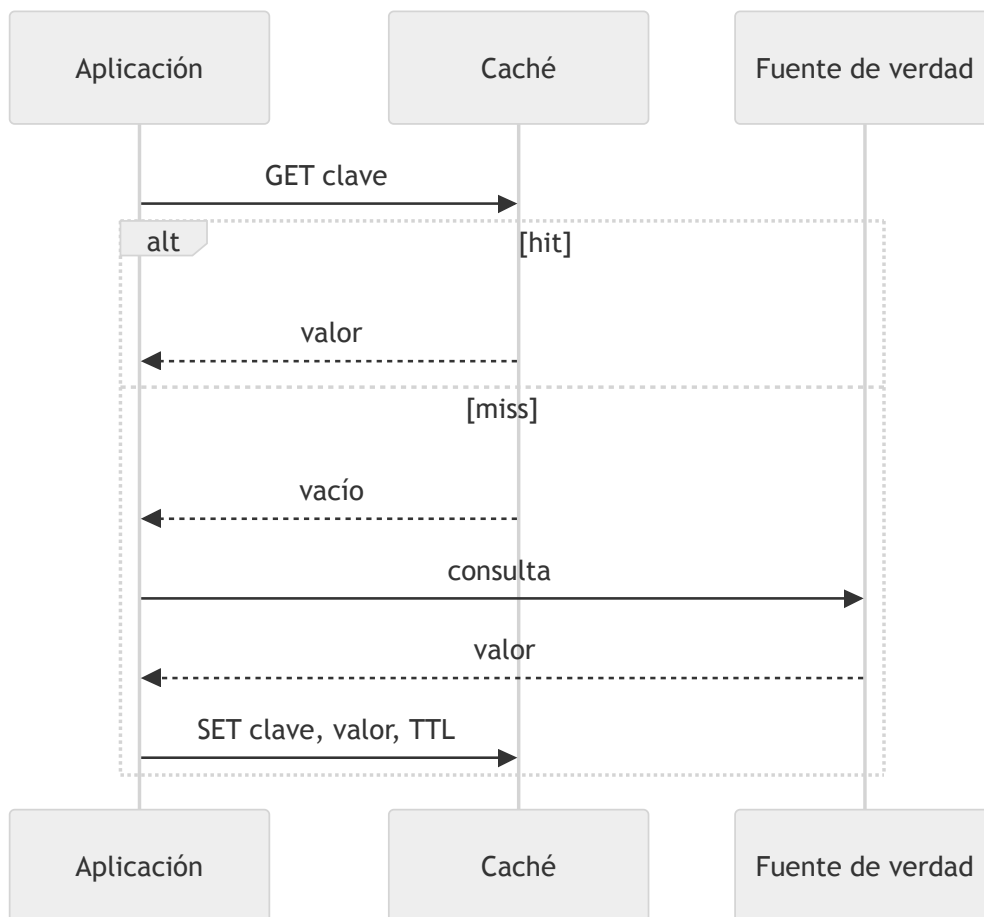
Una caché sin TTL ni control de capacidad tiende a convertirse en un almacenamiento accidental con comportamiento difícil de anticipar. Si un dato necesita conservarse con durabilidad, debe vivir en una fuente diseñada para esa responsabilidad, no depender de que una caché no lo expulse.

6. Patrones de acceso y actualización

Los patrones describen quién consulta la caché, quién la llena y qué ocurre al escribir. Ninguno es universal: se elige según la relación lectura/escritura, costo de reconstrucción, tolerancia al desfase y consecuencias de un fallo.

6.1 Cache-aside

En **cache-aside**, también llamado *lazy loading*, la aplicación controla la caché. Primero busca la clave; si hay miss, consulta la fuente de verdad, guarda el resultado y responde.



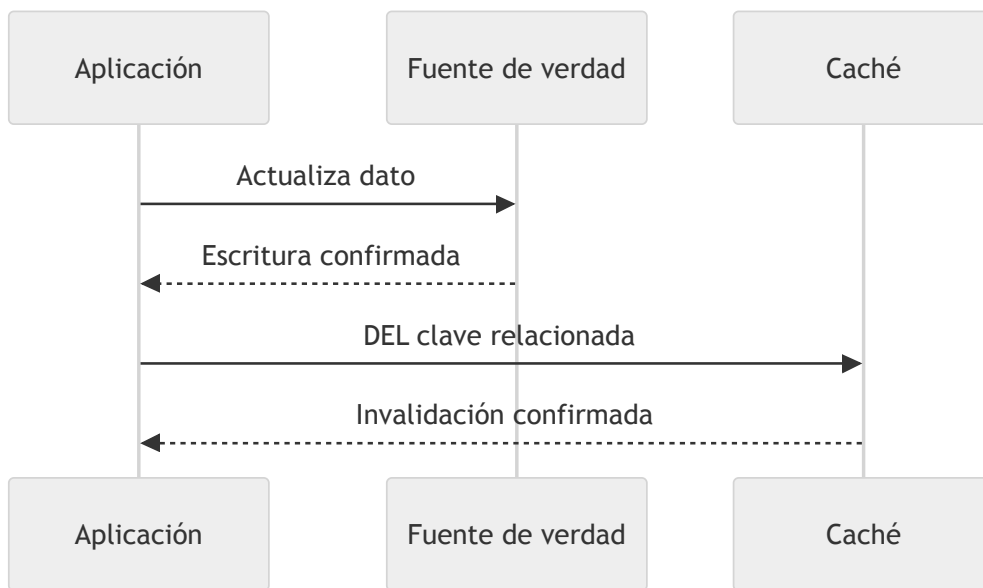
Ventajas:

- adopción incremental: solo se cachean lecturas elegidas;
- la caché se llena con datos que realmente se solicitan;
- la aplicación mantiene visible la decisión de recurrir a la fuente de verdad.

Costos y riesgos:

- el primer usuario tras un miss paga el costo completo;
- una entrada puede quedar vieja hasta su TTL o invalidación;
- varios misses simultáneos pueden reconstruir la misma clave en paralelo.

En escrituras, una estrategia habitual es: actualizar primero la fuente de verdad y, cuando esa operación se confirma, borrar la clave relacionada. La siguiente lectura la reconstruirá. Esta opción evita parte de los problemas de sincronizar dos copias, a cambio de un miss posterior.



6.2 Read-through

En **read-through**, la aplicación pide el dato a una capa de caché que conoce cómo obtenerlo de la fuente de verdad si hay miss. Para quien consume la caché, la lectura se ve como una sola operación; la responsabilidad de cargar queda encapsulada en esa capa.

El comportamiento observable se parece a cache-aside, pero cambia dónde vive el acoplamiento entre caché y fuente de verdad. Puede ser útil si una plataforma provee esa integración de manera controlada. Si la carga contiene lógica de negocio específica, esconderla dentro de infraestructura suele dificultar su prueba, evolución y observabilidad.

6.3 Write-through

En **write-through**, una escritura actualiza la fuente de verdad y la caché como parte del flujo síncrono. Así, el valor queda caliente para lecturas posteriores.

Como se señaló en 3.3, ambas actualizaciones no son atómicas por defecto. Si la base se actualiza pero la caché falla, el sistema debe decidir si devuelve éxito, reintenta, invalida la clave o degrada la operación. La fuente de verdad no debe quedar subordinada a la disponibilidad de la caché salvo que se haya aceptado explícitamente ese riesgo.

Es útil cuando el dato se lee inmediatamente después de escribirse y la menor probabilidad de miss justifica pagar más latencia y complejidad en la escritura.

6.4 Refresh-ahead

En **refresh-ahead**, una entrada popular se renueva antes de vencer. El objetivo es que la solicitud de un usuario no sea la que descubra el vencimiento y deba esperar la reconstrucción.

Es apropiado cuando el costo de cargar una clave es alto y el patrón de acceso es suficientemente predecible. Si se aplica sin criterio, refresca valores que nadie volverá a usar y vuelve a cargar la dependencia original sin aportar valor. Puede implementarse con trabajo en segundo plano, pero ese trabajo también necesita límites, timeouts y observabilidad.

6.5 Write-behind o write-back

En **write-behind**, la escritura llega primero a la caché y la persistencia en la fuente de verdad ocurre después, de forma asíncrona o agrupada. Su ventaja es reducir la latencia percibida y permitir agrupar escrituras.

El costo es mucho mayor que en los patrones anteriores: si la caché falla antes de persistir, los datos pueden perderse. También deben resolverse orden, reintentos, duplicados y visibilidad de valores todavía no durables. Solo se justifica si el dominio tolera una ventana de riesgo claramente acotada y la infraestructura está diseñada para minimizarla. No es apropiado para operaciones como confirmar una compra, registrar un pago o reservar una entrada única.

6.6 Comparación de patrones

Patrón	Quién carga o actualiza	Beneficio principal	Riesgo dominante	Complejidad y control
Cache-aside	La aplicación ante miss	Simplicidad y adopción selectiva	Miss inicial, invalidación	Baja-media; la aplicación controla el flujo
Read-through	La capa de caché ante miss	Encapsula la carga	Acoplamiento de infraestructura a la fuente	Media; el control se desplaza a la capa de caché
Write-through	Escritura síncrona en fuente y caché	Datos calientes tras escribir	Fallos parciales y mayor latencia de escritura	Media-alta; coordina dos destinos en el camino de escritura
Refresh-ahead	Proceso anticipado	Evita miss de claves populares	Trabajo inútil y más carga aguas abajo	Media-alta; requiere proceso de renovación y criterios de popularidad
Write-behind	Caché primero; fuente después	Baja latencia de escritura	Pérdida de datos y consistencia eventual	Alta; exige persistencia asíncrona, orden y reintentos

7. Invalidación y datos obsoletos

El problema de invalidación aparece porque un valor puede cambiar en la fuente de verdad mientras su copia sigue disponible. No hay una única política correcta: cada dato necesita una definición explícita de cuánto retraso tolera y qué acción corrige una copia vieja.

7.1 Estrategias habituales

- **TTL únicamente:** adecuado si el dato admite estar desactualizado durante toda la vida de la entrada y las escrituras son poco frecuentes.
- **Borrado al escribir:** luego de confirmar la escritura en la fuente de verdad, se elimina la clave; la próxima lectura reconstruye la copia. Es común en cache-aside.
- **Actualización al escribir:** se reemplaza el valor de caché junto con la escritura. Puede reducir misses, pero requiere tratar cuidadosamente los fallos parciales.
- **Versionado de claves:** una nueva representación usa una clave o versión nueva; las anteriores expiran naturalmente. Es útil al cambiar formatos.

La invalidación debe considerar dependencias. Modificar un evento puede afectar su detalle, una lista de eventos destacados y resultados de búsqueda. Escribir una única clave no alcanza si existen representaciones derivadas. Antes de cachear, conviene documentar qué operación invalida qué conjunto de claves.

7.2 Stale-while-revalidate

La estrategia **stale-while-revalidate** permite entregar temporalmente un valor conocido aunque haya vencido, mientras un proceso lo actualiza en segundo plano. Reduce la latencia y evita que muchos clientes esperen la misma recarga.

Solo es válida cuando servir una respuesta vieja por un período adicional es seguro. Es adecuada para una imagen o descripción pública; no para confirmar disponibilidad de un inventario. Debe definir dos límites: cuánto tiempo vale como dato fresco y cuánto tiempo adicional puede servirse como dato viejo.

8. Fallos, picos de tráfico y anti-patrones

8.1 Cache stampede

Un **cache stampede** —también llamado *thundering herd*— sucede cuando una clave popular vence y muchas solicitudes reciben miss al mismo tiempo. Todas intentan reconstruir el valor contra la misma dependencia; la caché deja de protegerla justamente durante el pico.

Las defensas posibles dependen del contexto:

- **exclusión por clave:** una sola solicitud reconstruye y las demás esperan o reciben un valor aceptable; si hay varias instancias, la coordinación debe ser distribuida;

- **dispersión de expiraciones** (*jitter*): no todas las claves similares vencen al mismo instante;
- **refresh-ahead**: renovar las claves populares antes de expirar;
- **stale-while-revalidate**: continuar sirviendo un valor vencido si el dominio lo permite;
- **límites y timeouts**: impedir que la reconstrucción agote conexiones o recursos de la fuente de verdad.

8.2 Cold start

Un **cold start** ocurre cuando la caché está vacía: tras un despliegue, reinicio, pérdida de un nodo o cambio de configuración. Una avalancha de misses puede concentrar la carga en la fuente de verdad. Precargar todas las claves tampoco es una respuesta automática: puede cargar datos que nadie usará y competir con el tráfico real.

El diseño debe elegir entre calentamiento selectivo de claves críticas, carga progresiva, límites de concurrencia y degradación aceptable. La estrategia depende de si la aplicación puede tolerar mayor latencia durante el arranque.

8.3 La caché puede fallar

Una caché distribuida es una dependencia de red. Puede responder lento, no responder, perder contenido o alcanzar su capacidad. La aplicación debe usar timeouts acotados y definir si puede continuar con la fuente de verdad. Ese fallback solo es seguro si la fuente puede absorber la carga adicional; de lo contrario, una caída de caché puede trasladar el incidente a la base de datos y producir un fallo en cascada.

No siempre se debe hacer fallback para todas las solicitudes. Según el caso, una degradación controlada puede ser preferible: responder un listado parcial, ocultar temporalmente una recomendación no esencial, servir un último valor conocido cuando su antigüedad sea aceptable o informar que una funcionalidad está temporalmente limitada. La elección depende del impacto de entregar menos información frente al impacto de sobrecargar la fuente de verdad. Las operaciones críticas deben conservar su validación contra la fuente autorizada, aun cuando la experiencia visible se degrade.

La caché no debe ser el único lugar donde existe información durable. Incluso en write-behind, la pérdida de datos es un riesgo explícito que debe ser aceptado y mitigado; no una consecuencia inesperada de un reinicio.

8.4 Anti-patrones frecuentes

Anti-patrón	Consecuencia	Corrección
Cachear todo por defecto	Memoria consumida y poco beneficio	Elegir datos por frecuencia, costo y frescura tolerada
Claves incompletas	Datos incorrectos o filtrados entre usuarios	Incluir todos los parámetros que cambian la respuesta
Sin TTL ni límite de capacidad	Crecimiento ilimitado y expulsiones impredecibles	Definir expiración, capacidad y política de eviction
Confiar en la caché para una decisión crítica	Operación basada en información vieja	Validar contra la fuente de verdad al confirmar
Fallback ilimitado a base de datos	La caída de caché derriba la fuente original	Timeouts, límites y degradación controlada
Ignorar métricas de evicción y misses	La caché falla silenciosamente como optimización	Observar resultados de negocio y operación

9. Observabilidad y decisión arquitectónica

Una caché debe poder justificarse con evidencia y operarse cuando sus supuestos dejan de cumplirse. Las métricas mínimas son:

- tasa de hits y misses, por operación o conjunto de claves;
- latencia de lectura y de reconstrucción tras un miss;
- errores, timeouts y disponibilidad de la caché;
- memoria utilizada, tamaño de valores y cantidad de claves;
- expiraciones y evicciones;
- carga y latencia de la fuente de verdad antes y después de incorporar la caché.

Una alta tasa de hits puede ocultar una mala decisión si, por ejemplo, se sirven datos demasiado viejos. De igual modo, una tasa menor puede ser suficiente si protege una consulta muy costosa. El efecto debe medirse de extremo a extremo: experiencia del usuario, consumo de la dependencia y corrección del dato.

Antes de incorporar una caché, un equipo debería responder:

1. ¿Qué operación es costosa y cuál es su fuente de verdad?
2. ¿Quién necesita el dato y con qué frecuencia se repite?
3. ¿Cuánto tiempo puede estar desactualizado?
4. ¿Qué evento invalida o actualiza cada clave derivada?
5. ¿Qué patrón de lectura y escritura corresponde?

6. ¿Qué ocurre ante miss, expiración, evicción y caída de la caché?
 7. ¿Qué métricas demostrarán que la decisión funciona?
-

10. EventOS: aplicación práctica

En una plataforma de venta de entradas, el catálogo público y el detalle descriptivo de un evento son buenos candidatos para cache-aside: reciben muchas lecturas y cambian con una frecuencia relativamente baja. Una clave como `eventos:42:publico:v1` puede almacenar la representación pública con un TTL acotado.

Cuando el productor modifica nombre, fecha, sede o imagen, la aplicación invalida esa clave y las listas derivadas afectadas.

La disponibilidad transaccional de entradas merece otro tratamiento. Una página puede mostrar una cantidad aproximada almacenada para mejorar la experiencia, pero la confirmación de compra debe consultar y reservar contra el componente dueño del inventario. De ese modo, una copia retrasada no permite vender dos veces la misma entrada.

Ante un pico de ventas, el sistema debe anticipar qué ocurre cuando una clave popular vence. Usar dispersión de TTL, renovación anticipada o stale-while-revalidate para datos no críticos puede evitar que miles de misses consulten simultáneamente la base. Ninguna de esas estrategias reemplaza la validación fuerte del inventario en el momento de la operación de compra.

11. Trade-offs y decisiones arquitectónicas

Decisión	Se gana	Se paga	Conviene cuando
No usar caché	Menos componentes y datos siempre leídos desde origen	Más latencia y carga repetida	La fuente soporta la carga o el dato exige máxima actualidad
Caché local	Latencia mínima y simplicidad	Copias divergentes entre réplicas	El valor es pequeño, efímero y tolera desfase
Caché distribuida	Reutilización entre instancias	Red, operación y nueva dependencia	Muchas réplicas comparten lecturas costosas
TTL largo	Más hits y menor carga	Mayor ventana de desactualización	El dominio tolera información vieja
TTL corto o invalidación	Datos más frescos	Más misses y complejidad	Un cambio debe reflejarse pronto
Write-through	Datos calientes tras escribir	Más latencia y manejo de fallos parciales	Hay lectura inmediata posterior y se justifica el costo
Write-behind	Escrituras rápidas	Riesgo de pérdida y consistencia eventual	El dominio acepta explícitamente esa ventana de riesgo

La pregunta no es si una caché “acelera” el sistema en abstracto. La pregunta es qué lectura concreta se quiere optimizar, qué propiedad de frescura se está dispuesto a relajar y cómo se preserva el comportamiento correcto cuando la caché no cumple su función.

12. Puntos clave

- Una caché guarda copias temporales; la fuente de verdad conserva la autoridad sobre el dato.
- Un hit reduce trabajo, pero un miss todavía necesita la fuente original; la caché no elimina esa dependencia.
- El TTL acota la vida de una copia, pero no reemplaza la invalidación cuando un cambio debe reflejarse antes.
- Las claves deben incluir todos los parámetros que alteran una respuesta para evitar datos incorrectos o filtraciones.
- La caché local es muy rápida, pero cada instancia conserva su propia copia; una distribuida comparte datos a cambio de red y operación.

- Cache-aside es un patrón simple y frecuente; write-through y write-behind incorporan riesgos adicionales en las escrituras.
- Las políticas de expulsión y la capacidad importan: una entrada válida puede ser expulsada antes de vencer.
- Cache stampede, cold start y caída de caché deben diseñarse y observarse; no son casos excepcionales irrelevantes.
- La disponibilidad visible de un recurso puede cachearse, pero una operación crítica debe validar contra la fuente de verdad.
- La decisión correcta equilibra rendimiento, costo, frescura, complejidad y comportamiento ante fallos.